

NestUp BD

Complete Code Analysis and Architecture Documentation

Property Rental Platform for Bangladesh

Software Project Lab - 2

Department of Computer Science and Engineering

Islamic University of Technology

August 10, 2025

Contents

1	Executive Summary	4
2	Project Overview	4
2.1	Technology Stack	4
2.2	Key Features	4
3	Project Structure	5
3.1	Architecture Principles	5
4	Backend Architecture	5
4.1	Server Entry Point	5
4.1.1	Key Functionalities	6
4.1.2	Server Initialization Flow	6
4.2	Database Models	6
4.2.1	User Model	6
4.2.2	Service Model	7
4.2.3	Booking Model	8
4.2.4	Payment Model	9
4.3	API Routes	10
4.3.1	Authentication Routes	10
4.3.2	Main API Routes	11
4.4	Middleware	12
4.4.1	Authentication Middleware	12
4.4.2	File Upload Middleware	13
5	Frontend Architecture	13
5.1	Application Entry Point	13
5.2	Main App Component	14
5.3	Context Management	14
5.3.1	Authentication Context	14
5.3.2	4.4 Page Components (Client/src/pages/)	15
5.3.3	4.5 Reusable Components (Client/src/components/)	16
5.3.4	4.6 Utility Functions (Client/src/utills/)	17
5.4	5. DATABASE IMPLEMENTATION	17
5.4.1	5.1 MongoDB Connection	17
5.4.2	5.2 Database Schema Design	17
5.4.3	5.3 Data Validation	18
5.5	6. API ENDPOINTS AND DATA FLOW	18
5.5.1	6.1 Authentication Flow	18
5.5.2	6.2 Service Management Flow	18
5.5.3	6.3 Booking and Payment Flow	18
5.5.4	6.4 Wishlist Management	19
5.6	7. AUTHENTICATION SYSTEM	19
5.6.1	7.1 JWT Implementation	19
5.6.2	7.2 Password Security	19

5.6.3	7.3 Route Protection	19
5.7	8. PAYMENT SYSTEM	19
5.7.1	8.1 Payment Methods	19
5.7.2	8.2 Payment Processing	19
5.7.3	8.3 Payment History	20
5.8	9. FILE-BY-FILE ANALYSIS	20
5.8.1	9.1 Backend Files	20
5.8.2	9.2 Frontend Files	21
5.9	10. FRONTEND-BACKEND CONNECTION FLOW	22
5.9.1	10.1 API Communication Pattern	22
5.9.2	10.2 Authentication Integration	22
5.9.3	10.3 Error Handling	22
5.9.4	10.4 Data Transformation	23
5.10	11. CORE FUNCTIONALITIES	23
5.10.1	11.1 User Management	23
5.10.2	11.2 Property Management	23
5.10.3	11.3 Booking System	23
5.10.4	11.4 Payment Processing	23
5.10.5	11.5 Wishlist System	23
5.11	12. MONGODB COLLECTIONS AND SCHEMA	24
5.11.1	12.1 Collection Initialization	24
5.11.2	12.2 Collection Details	24
5.11.3	12.3 Data Relationships	25
5.11.4	12.4 Database Operations	25
5.12	CONCLUSION	25

1 Executive Summary

This document provides a comprehensive technical analysis of the NestUp BD platform, a modern property rental solution designed specifically for the Bangladeshi market. The platform serves students and professionals seeking accommodation through an intuitive web-based interface that facilitates property listing, searching, booking, and secure payment processing.

2 Project Overview

NestUp BD is a comprehensive property rental platform designed specifically for students and professionals in Bangladesh. The application facilitates property listing, searching, booking, and payment processing with a modern web-based interface.

2.1 Technology Stack

The platform is built using modern web technologies to ensure scalability, maintainability, and optimal performance:

Component	Technology
Frontend Framework	React.js with Vite build tool
Backend Framework	Node.js with Express.js
Database	MongoDB with Mongoose ODM
Authentication	JWT (JSON Web Tokens)
File Upload	Multer middleware
Styling	CSS with custom variables and responsive design
Development Tools	ESLint, Prettier, Nodemon
Version Control	Git with GitHub

Table 1: Technology Stack Overview

2.2 Key Features

- **Multi-tenant Architecture:** Support for property owners and tenants
- **Advanced Search:** Location-based filtering with multiple criteria
- **Secure Payments:** Multiple payment methods with transaction tracking
- **Real-time Updates:** Live availability and booking status
- **Mobile Responsive:** Optimized for all device types
- **Admin Dashboard:** Comprehensive management interface

3 Project Structure

The project follows a well-organized modular structure that separates concerns between frontend and backend components:

```

1 NestUp BD/
2   Client/                                # Frontend React application
3     src/
4       components/                        # Reusable UI components
5       pages/                             # Main application pages
6       context/                           # React Context for
state management
7       utils/                             # Utility functions
8       assets/                             # Static assets
9       public/                             # Public static files
10      package.json                       # Frontend dependencies
11      vite.config.js                     # Vite configuration
12   server/                               # Backend Node.js application
13     models/                             # MongoDB schemas and
models
14     routes/                             # API route handlers
15     middleware/                         # Custom middleware
functions
16     uploads/                             # File upload storage
17     server.js                           # Main server entry point
18     package.json                       # Backend dependencies
19     .env                               # Environment variables
20   Docs/                                 # Documentation files
21   test-payment.js                       # Payment testing script

```

Listing 1: Project Directory Structure

3.1 Architecture Principles

- **Separation of Concerns:** Clear distinction between frontend and backend responsibilities
- **Modular Design:** Components are organized into logical modules for maintainability
- **RESTful API:** Standard HTTP methods and status codes for API communication
- **Security First:** JWT authentication and input validation at all levels
- **Scalable Structure:** Organized codebase that supports future enhancements

4 Backend Architecture

4.1 Server Entry Point

File: `server/server.js`

Purpose: Main application server that initializes the Express app, connects to MongoDB, sets up middleware, and starts the server.

4.1.1 Key Functionalities

- Express server initialization and configuration
- MongoDB connection using Mongoose ODM
- CORS configuration for cross-origin requests
- Cookie parser for JWT token handling
- Static file serving for uploaded content
- API route mounting and error handling
- Server startup on port 3000

4.1.2 Server Initialization Flow

1. Load environment variables from `.env` file
2. Create Express application instance
3. Configure middleware (CORS, JSON parser, cookie parser)
4. Establish MongoDB database connection
5. Mount API routes and middleware
6. Start server and listen on specified port

4.2 Database Models

Location: *server/models/*

The application uses MongoDB with Mongoose ODM for data modeling. Each model defines the schema, validation rules, and business logic for different entities.

4.2.1 User Model

File: `User.js`

Purpose: Defines the user schema for authentication and profile management.

Field	Type	Constraints
name	String	Required, trimmed, min 2 chars
email	String	Required, unique, lowercase, email validation
password	String	Required, min 6 chars, hashed
role	Enum	['user', 'admin'], default: 'user'
isVerified	Boolean	Default: true
profile.phone	String	Optional
profile.nidNumber	String	Optional
profile.dateOfBirth	Date	Optional
profile.gender	Enum	['male', 'female', 'other']
profile.occupation	String	Optional
profile.emergencyContact	Object	name and phone
profile.address	Object	street, city, district, postalCode
preferences	Object	User settings
createdAt, updatedAt	Timestamps	Auto-generated

Table 2: User Model Schema

Schema Fields

Methods and Middleware

- `comparePassword()`: Compares plain text password with hashed password
- **Pre-save middleware**: Automatically hashes password before saving

4.2.2 Service Model

File: `Service.js`

Purpose: Defines the property/service schema for rental listings.

Field	Type	Constraints
title	String	Required, max 200 chars
propertyType	Enum	['apartment', 'house', 'room', 'hostel', 'dormitory', 'sublet']
description	String	Required, max 2000 chars
location.district	String	Required
location.area	String	Required
location.address	String	Required
location.coordinates	Object	latitude and longitude
price	Number	Required, min 0
availability.from	Date	Required
availability.to	Date	Required
availability.isAvailable	Boolean	Default: true

Table 3: Service Model - Core Fields

Core Schema Fields

Field	Type	Description
propertyDetails.bedrooms	String	Number of bedrooms
propertyDetails.bathrooms	String	Number of bathrooms
propertyDetails.squareFeet	Number	Property size
propertyDetails.furnishing	Enum	['unfurnished', 'semi-furnished', 'fully-furnished']
amenities	Object	Boolean flags for various amenities
images	Array	Image URLs
thumbnail	String	Main image URL
contact	Object	name, phone, email, whatsapp
owner	ObjectId	Reference to User
rating	Object	average and count
reviews	Array	Review objects
isVerified	Boolean	Default: false
isActive	Boolean	Default: true

Table 4: Service Model - Property Details

Property Details and Features

Methods and Indexes

- **Pre-save middleware:** Updates the updatedAt field automatically
- **Database Indexes:** Created on location fields and price for efficient querying
- **Virtual Properties:** Computed fields for enhanced functionality

4.2.3 Booking Model

File: `Booking.js`

Purpose: Manages property booking information and status.

Field	Type	Constraints
service	ObjectId	Reference to Service (required)
user	ObjectId	Reference to User (required)
startDate	Date	Required
endDate	Date	Required
duration	Object	days and months
basePrice	Number	Required
totalAmount	Number	Required
fees	Object	service fee, cleaning fee, security deposit
status	Enum	['pending', 'confirmed', 'active', 'completed', 'cancelled', 'rejected']
paymentStatus	Enum	['pending', 'partial', 'paid', 'refunded', 'failed']

Table 5: Booking Model - Core Fields

Core Schema Fields

Field	Type	Description
guestInfo	Object	numberOfGuests and specialRequests
contactInfo	Object	phone and email (required)
confirmationCode	String	Auto-generated unique identifier
confirmedAt	Date	Timestamp when booking was confirmed
createdAt	Date	Auto-generated creation timestamp
updatedAt	Date	Auto-updated modification timestamp

Table 6: Booking Model - Additional Fields

Additional Information Fields

Methods and Business Logic

- **confirm()**: Updates status to confirmed and sets confirmedAt timestamp
- **checkAvailability()**: Static method to check date conflicts with existing bookings
- **Pre-save middleware**: Automatically calculates duration and generates confirmation code
- **Virtual property**: durationInDays for easy access to booking duration
- **Status management**: Comprehensive booking lifecycle tracking

4.2.4 Payment Model

File: `Payment.js`

Purpose: Handles payment processing and transaction records.

Field	Type	Constraints
booking	ObjectId	Reference to Booking (required)
user	ObjectId	Reference to User (required)
service	ObjectId	Reference to Service (required)
amount	Number	Required, min 0
currency	String	Default: 'BDT'
paymentMethod	Enum	['mobile_banking', 'bank_transfer', 'credit_card', 'cash']
paymentDetails	Object	Method-specific payment details
status	Enum	['pending', 'processing', 'completed', 'failed', 'refunded', 'cancelled']

Table 7: Payment Model - Core Fields

Core Transaction Fields

Field	Type	Description
personalInfo	Object	User's personal information for payment
termsAccepted	Boolean	Required consent flag
processedAt	Date	Timestamp when payment was processed
failureReason	String	Reason for payment failure
gatewayResponse	Object	Payment gateway response data
refund	Object	Refund details and status
createdAt	Date	Auto-generated creation timestamp
updatedAt	Date	Auto-updated modification timestamp

Table 8: Payment Model - Additional Fields

Additional Transaction Data

Payment Processing Methods

- **markAsCompleted()**: Updates status to completed and sets processedAt timestamp
- **markAsFailed()**: Sets status to failed with detailed failure reason
- **processRefund()**: Handles refund processing and status updates
- **getPaymentStats()**: Static method for payment analytics and reporting
- **Gateway Integration**: Seamless integration with multiple payment gateways

4.3 API Routes

Directory: **server/routes/**

Purpose: Defines RESTful API endpoints for client-server communication.

4.3.1 Authentication Routes

File: **auth.js**

Purpose: Handles user authentication, registration, and session management.

Method	Endpoint	Purpose	Protection
POST	/register	User registration with validation	Public
POST	/login	User login with JWT token generation	Public
POST	/logout	User logout with token clearing	Public
GET	/me	Get current user profile	Protected
PUT	/profile	Update user profile	Protected

Table 9: Authentication API Endpoints

API Endpoints

Authentication Flow

1. User submits credentials through registration or login form
2. Server validates input data and checks business rules
3. For login: verify password hash and generate JWT token
4. For registration: hash password, create user, and generate token
5. Set HTTP-only cookie with JWT token for secure session management
6. Return user data (excluding sensitive information like password)
7. Implement proper error handling for invalid credentials or duplicate users

4.3.2 Main API Routes

File: `api.js`

Purpose: Core application API endpoints for services, bookings, and payments.

Method	Endpoint	Purpose	Protection
GET	/services	Fetch all services with filtering and pagination	Public
GET	/services/:id	Get specific service details	Public
POST	/services	Create new service	Protected
PUT	/services/:id	Update service	Protected (owner only)
DELETE	/services/:id	Delete service	Protected (owner only)

Table 10: Service Management API Endpoints

Service Management Endpoints

Method	Endpoint	Purpose	Protection
POST	/bookings	Create new booking	Protected
GET	/bookings	Get user's bookings	Protected
GET	/bookings/:id	Get specific booking	Protected
PUT	/bookings/:id/confirm	Confirm booking	Protected
PUT	/bookings/:id/cancel	Cancel booking	Protected

Table 11: Booking Management API Endpoints

Booking Management Endpoints

Method	Endpoint	Purpose	Protection
POST	/payments	Process payment	Protected
GET	/payments/:paymentId	Get payment status	Protected
GET	/my-payments	Get user's payment history	Protected

Table 12: Payment Processing API Endpoints

Payment Processing Endpoints

Method	Endpoint	Purpose	Protection
POST	/wishlist/add	Add service to wishlist	Protected
DELETE	/wishlist/remove/:serviceId	Remove from wishlist	Protected
GET	/wishlist	Get user's wishlist	Protected
GET	/wishlist/check/:serviceId	Check if service is in wishlist	Protected

Table 13: Wishlist Management API Endpoints

Wishlist Management Endpoints

4.4 Middleware

Directory: **server/middleware/**

Purpose: Implements cross-cutting concerns like authentication, validation, and error handling.

4.4.1 Authentication Middleware

File: **auth.js**

Purpose: Verifies JWT tokens and protects routes from unauthorized access.

Key Features

- **Token Verification:** Validates JWT tokens from HTTP-only cookies
- **User Context:** Attaches authenticated user information to request object
- **Route Protection:** Prevents unauthorized access to protected endpoints
- **Error Handling:** Provides clear error messages for authentication failures

Authentication Functions

- **verifyToken():** Extracts and verifies JWT from cookies or Authorization headers
- **checkAuth():** Ensures user exists and attaches user object to request
- **requireAuth():** Middleware wrapper for protected routes

Authentication Process

1. Extract JWT token from HTTP-only cookies or Authorization header
2. Verify token signature using JWT secret key
3. Decode token to extract user ID and other claims
4. Query database to find user by decoded token ID
5. Attach complete user object to request for downstream middleware
6. Call next middleware in chain or return authentication error

4.4.2 File Upload Middleware

File: `upload.js`

Purpose: Handles file uploads using Multer middleware for property images.

Setting	Value
Storage Type	Local disk storage
Upload Directory	uploads/
File Naming	Timestamp + original name
File Types	Images only (jpg, jpeg, png, gif)
Size Limit	5MB per file
Field Name	'images' (multiple files)

Table 14: File Upload Configuration

Upload Configuration

5 Frontend Architecture

Technology Stack: React 18, React Router, Context API, CSS Modules

Architecture Pattern: Component-based architecture with centralized state management

5.1 Application Entry Point

File: `Client/src/main.jsx`

Purpose: React application initialization and root component rendering.

Core Functionality

- **React Initialization:** Imports React 18 and ReactDOM for modern rendering
- **Root Rendering:** Renders the main App component using createRoot API
- **DOM Mounting:** Mounts application to DOM element with id 'root'
- **Strict Mode:** Enables React.StrictMode for development checks

5.2 Main App Component

File: `Client/src/App.jsx`

Purpose: Root component that orchestrates routing and global providers.

Application Structure

- **Context Providers:** Wraps application in AuthProvider for global authentication state
- **Routing Setup:** Configures React Router with BrowserRouter for client-side navigation
- **Route Definitions:** Defines all application routes and their corresponding components
- **Global Styles:** Imports and applies global CSS styles and theme configurations
- **Error Boundaries:** Implements error handling for robust user experience

5.3 Context Management

Directory: `Client/src/context/`

Purpose: Centralized state management using React Context API.

5.3.1 Authentication Context

File: `AuthContext.js`

Purpose: Global state management for user authentication and session handling.

Property	Type	Description
user	Object/null	Current authenticated user object
loading	Boolean	Authentication check status
isAuthenticated	Boolean	User authentication state
error	String/null	Authentication error messages

Table 15: Authentication Context State

State Properties

Method	Parameters	Purpose
login()	credentials	Handles user login process
logout()	none	Handles user logout and cleanup
checkAuth()	none	Verifies current authentication status
register()	userData	Handles new user registration
updateProfile()	profileData	Updates user profile information

Table 16: Authentication Context Methods

Authentication Methods

Key Features

- **Persistent Sessions:** Maintains authentication state across page refreshes
- **Global Access:** Provides authentication methods to all components via useAuth hook
- **Loading States:** Manages loading indicators during authentication operations
- **Token Management:** Handles JWT token storage and retrieval securely
- **Error Handling:** Comprehensive error management for authentication failures

5.3.2 4.4 Page Components (Client/src/pages/)

4.4.1 Home Page (Home/Home.jsx) **Purpose:** Landing page with hero section and service overview.

Components Used: - Header: Navigation and user menu - HeroSection: Main banner with search functionality - ServiceCards: Available service categories - HowItWorks: Process explanation - TrustBadges: Trust indicators - Footer: Site footer with links

4.4.2 Authentication Pages **Login Page (Login/Login.jsx):** - User login form with email and password - Form validation and error handling - Redirects to dashboard on successful login - Links to registration page

Register Page (Register/Register.jsx): - User registration form with multiple fields - Password confirmation validation - Terms and conditions acceptance - Automatic login after successful registration

4.4.3 Search Page (Search/Search.jsx) **Purpose:** Property search and listing display.

Key Features: - Fetches services from backend API - Implements filtering by location, price, amenities - Displays results in grid layout using ListingCard components - Handles loading states and empty results - Integrates with ServiceModal for detailed views

Data Flow: 1. Fetch services from /api/services endpoint 2. Transform backend data to frontend format 3. Apply client-side filters 4. Render filtered results 5. Handle user interactions (view details, book, save)

4.4.4 Payment Page (Payment/Payment.jsx) **Purpose:** Multi-step payment process for property bookings.

Step Structure: 1. Personal Information: User details collection 2. Payment Method: Payment option selection 3. Confirmation: Review and submit

Payment Flow: 1. Validate service ID and user authentication 2. Create booking via POST /api/bookings 3. Process payment via POST /api/payments 4. Handle success/error responses 5. Navigate to dashboard on completion

Components: - PaymentStepIndicator: Progress visualization - PersonalInfoStep: Personal details form - PaymentMethodStep: Payment method selection - ConfirmationStep: Final review - PaymentSidebar: Booking summary - FormNavigation: Step navigation controls

4.4.5 Dashboard Page (Dashboard/Dashboard.jsx) **Purpose:** User dashboard with multiple views and functionalities.

Dashboard Views: - Overview: Summary statistics and recent activity - MyListings: User's property listings management - PaymentHistory: Transaction history and details - Wishlist: Saved properties - Profile: User profile management

State Management: - activeView: Current dashboard section - userData: User profile information - listings: User's property listings - payments: Payment transaction history - wishlist: Saved properties list

4.4.6 Service Provision Page (ProvideService/ProvideService.jsx) **Purpose:** Multi-step form for property listing creation.

Form Steps: 1. Basic Information: Title, type, description 2. Location Details: Address and area information 3. Pricing & Availability: Rent and date ranges 4. Property Details: Bedrooms, bathrooms, features 5. Amenities & Features: Available facilities 6. Photos & Contact: Images and contact information 7. Review & Submit: Final review before submission

Form Management: - Comprehensive form validation - Step-by-step navigation - Data persistence across steps - Image upload handling - Error handling and user feedback

5.3.3 4.5 Reusable Components (Client/src/components/)

4.5.1 Header Component (Header/Header.jsx) **Purpose:** Main navigation bar with authentication-aware menu.

Features: - Responsive navigation menu - User authentication status display - Drop-down menu for authenticated users - Mobile-friendly hamburger menu - Logo and brand navigation

4.5.2 ListingCard Component (ListingCard/ListingCard.jsx) **Purpose:** Property listing display card with interactive features.

Features: - Property image display with fallback - Property details (title, location, price, availability) - Action buttons (View Details, Book, Save) - Wishlist integration - Owner listing detection - Responsive design

Interaction Flow: - View Details: Opens ServiceModal or navigates to payment - Book: Navigates to payment page with property data - Save: Adds/removes from wishlist via API

4.5.3 ServiceModal Component (ServiceModal/ServiceModal.jsx) **Purpose:** Detailed property information modal dialog.

Content Sections: - Image gallery with main image and thumbnails - Property details (type, location, features) - Availability and pricing information - Amenities grid with icons - Property description - Contact information - Action buttons

4.5.4 FilterSidebar Component (FilterSidebar/FilterSidebar.jsx) **Purpose:** Search filtering interface for property listings.

Filter Options: - Location filters (district, area) - Price range (minimum and maximum) - Date range (availability period) - Property features (verified hosts, hygiene badge) - Real-time filter application

5.3.4 4.6 Utility Functions (Client/src/utils/)

4.6.1 Validation Utilities (validation.js) **Purpose:** Form validation functions for payment and other forms.

Validation Functions: - validateStep(): Step-specific validation for payment form - Email validation with regex patterns - Phone number validation for Bangladesh format - Required field validation - Payment method specific validation

=====

5.4 5. DATABASE IMPLEMENTATION

5.4.1 5.1 MongoDB Connection

Location: server/server.js **Connection String:** mongodb://localhost:27017/nestup_bd
ODM: Mongoose for schema definition and data modeling

Connection Process: 1. Load MongoDB URI from environment variables 2. Use mongoose.connect() with connection options 3. Handle connection events (connected, error, disconnected) 4. Implement graceful shutdown on process termination

5.4.2 5.2 Database Schema Design

5.2.1 Collections Overview

- **users:** User accounts and profiles
- **services:** Property listings and details
- **bookings:** Booking requests and status
- **payments:** Payment transactions and history

5.2.2 Relationships

- User → Services (One-to-Many): User can own multiple properties
- Service → Bookings (One-to-Many): Property can have multiple bookings
- User → Bookings (One-to-Many): User can make multiple bookings
- Booking → Payment (One-to-One): Each booking has one payment
- User → Payments (One-to-Many): User can have multiple payments

5.2.3 Indexes

- Users: email (unique), createdAt
- Services: location.district, location.area, price, createdAt
- Bookings: user, service, startDate, endDate
- Payments: user, booking, status, createdAt

5.4.3 5.3 Data Validation

Schema-Level Validation: - Required fields enforcement - Data type validation - Enum value restrictions - String length limits - Number range validation - Email format validation

Application-Level Validation: - Business logic validation - Cross-field validation - Custom validation functions - Error message customization

=====

5.5 6. API ENDPOINTS AND DATA FLOW

5.5.1 6.1 Authentication Flow

Registration Process: 1. Frontend: User submits registration form 2. Frontend: POST /api/auth/register with user data 3. Backend: Validate input data 4. Backend: Hash password using bcrypt 5. Backend: Create user in database 6. Backend: Generate JWT token 7. Backend: Set HTTP-only cookie with token 8. Backend: Return user data (excluding password) 9. Frontend: Update AuthContext with user data 10. Frontend: Redirect to dashboard

Login Process: 1. Frontend: User submits login form 2. Frontend: POST /api/auth/login with credentials 3. Backend: Find user by email 4. Backend: Compare password with stored hash 5. Backend: Generate JWT token if valid 6. Backend: Set HTTP-only cookie with token 7. Backend: Return user data 8. Frontend: Update AuthContext 9. Frontend: Redirect to intended page

5.5.2 6.2 Service Management Flow

Service Creation: 1. Frontend: User fills ProvideService form 2. Frontend: Upload images via FormData 3. Frontend: POST /api/services with service data 4. Backend: Validate user authentication 5. Backend: Process uploaded images 6. Backend: Create service in database 7. Backend: Return created service data 8. Frontend: Show success message and redirect

Service Search: 1. Frontend: User applies filters on Search page 2. Frontend: GET /api/services with query parameters 3. Backend: Build MongoDB filter object 4. Backend: Execute paginated query with population 5. Backend: Return services array with pagination info 6. Frontend: Transform data for display 7. Frontend: Render ListingCard components

5.5.3 6.3 Booking and Payment Flow

Complete Booking Process: 1. Frontend: User clicks “Book” on ListingCard 2. Frontend: Navigate to Payment page with property data 3. Frontend: User completes payment form steps 4. Frontend: POST /api/bookings to create booking 5. Backend: Validate service exists and availability 6. Backend: Check for booking conflicts 7. Backend: Calculate total amount and fees 8. Backend: Create booking with ‘pending’ status 9. Backend: Return booking data with ID 10. Frontend: POST /api/payments with booking ID 11. Backend: Validate payment details 12. Backend: Create payment record 13. Backend: Update booking payment status 14. Backend: Return payment confirmation 15. Frontend: Show success message 16. Frontend: Navigate to dashboard

5.5.4 6.4 Wishlist Management

Add to Wishlist: 1. Frontend: User clicks “Save” on ListingCard 2. Frontend: POST /api/wishlist/add with serviceId 3. Backend: Validate authentication and service 4. Backend: Check if already in wishlist 5. Backend: Add to user’s wishlist array 6. Backend: Return success response 7. Frontend: Update UI state

=====

5.6 7. AUTHENTICATION SYSTEM

5.6.1 7.1 JWT Implementation

Token Generation: - Payload: { userId, email, iat, exp } - Secret: Stored in environment variables - Expiration: 7 days - Storage: HTTP-only cookies for security

Token Verification: - Middleware: verifyToken() function - Extraction: From cookies or Authorization header - Validation: JWT signature and expiration - User Lookup: Find user by decoded ID - Request Attachment: Add user object to req.user

5.6.2 7.2 Password Security

Hashing: - Algorithm: bcrypt with salt rounds (10) - Timing: Pre-save middleware in User model - Comparison: comparePassword() method - Storage: Never store plain text passwords

5.6.3 7.3 Route Protection

Protected Routes: - All routes requiring authentication use verifyToken middleware - Additional checkAuth middleware ensures user exists - Role-based access control for admin functions - Owner-only access for service management

=====

5.7 8. PAYMENT SYSTEM

5.7.1 8.1 Payment Methods

Supported Methods: - Mobile Banking (bKash, Nagad, Rocket) - Bank Transfer - Credit/Debit Card - Cash Payment

Method-Specific Data: - Mobile Banking: Phone number, transaction ID - Bank Transfer: Bank name, account number - Credit Card: Card number, expiry, CVV, holder name - Cash: Contact information for arrangement

5.7.2 8.2 Payment Processing

Payment States: - pending: Initial state after creation - processing: Payment being processed - completed: Payment successful - failed: Payment failed with reason - refunded: Payment refunded - cancelled: Payment cancelled by user

Processing Flow: 1. Create booking first 2. Validate payment details 3. Create payment record 4. Process payment (simulated) 5. Update payment status 6. Update booking payment status 7. Send confirmation

5.7.3 8.3 Payment History

Data Tracking: - All payment transactions stored - Status change history - Payment method details - Associated booking information - Refund tracking

Dashboard Integration: - Payment history view in dashboard - Filtering by status and date - Detailed payment information - Download/export capabilities

=====

5.8 9. FILE-BY-FILE ANALYSIS

5.8.1 9.1 Backend Files

server/server.js **Purpose:** Main server application entry point **Key Functions:** - Express app initialization - Middleware configuration - Database connection - Route mounting - Server startup **Dependencies:** express, mongoose, cors, cookie-parser, dotenv **Exports:** None (entry point)

server/models/User.js **Purpose:** User data model and schema **Key Functions:** - User schema definition - Password hashing middleware - Password comparison method - Profile data structure **Dependencies:** mongoose, bcryptjs **Exports:** User model

server/models/Service.js **Purpose:** Property service data model **Key Functions:** - Service schema definition - Location and pricing structure - Amenities and features - Review system integration **Dependencies:** mongoose **Exports:** Service model

server/models/Booking.js **Purpose:** Booking management data model **Key Functions:** - Booking schema definition - Date validation - Status management - Availability checking **Dependencies:** mongoose **Exports:** Booking model

server/models/Payment.js **Purpose:** Payment processing data model **Key Functions:** - Payment schema definition - Payment method handling - Status tracking - Refund management **Dependencies:** mongoose **Exports:** Payment model

server/routes/auth.js **Purpose:** Authentication route handlers **Key Functions:** - User registration - User login/logout - Profile management - JWT token handling **Dependencies:** express, bcryptjs, jsonwebtoken, User model **Exports:** Express router

server/routes/api.js **Purpose:** Main API route handlers **Key Functions:** - Service CRUD operations - Booking management - Payment processing - Wishlist operations **Dependencies:** express, all models, auth middleware **Exports:** Express router

server/middleware/auth.js **Purpose:** Authentication middleware **Key Functions:** - JWT token verification - User authentication checking - Request user attachment **Dependencies:** jsonwebtoken, User model **Exports:** verifyToken, checkAuth functions

server/middleware/upload.js **Purpose:** File upload middleware **Key Functions:** - Multer configuration - File storage setup - File type filtering - Size limitations **Dependencies:** multer, path **Exports:** upload middleware

5.8.2 9.2 Frontend Files

Client/src/main.jsx **Purpose:** React application entry point **Key Functions:** - React DOM rendering - App component mounting **Dependencies:** react, react-dom, App component **Exports:** None (entry point)

Client/src/App.jsx **Purpose:** Root application component **Key Functions:** - Router setup - Global providers - Route definitions **Dependencies:** react-router-dom, AuthProvider, all page components **Exports:** App component

Client/src/context/AuthContext.js **Purpose:** Authentication context provider **Key Functions:** - Global auth state management - Login/logout functions - Authentication checking **Dependencies:** react, createContext, useState, useEffect **Exports:** AuthProvider, useAuth hook

Client/src/pages/Home/Home.jsx **Purpose:** Landing page component **Key Functions:** - Hero section display - Service overview - Navigation integration **Dependencies:** react, Header, HeroSection, ServiceCards, etc. **Exports:** Home component

Client/src/pages/Login/Login.jsx **Purpose:** User login page **Key Functions:** - Login form handling - Authentication API calls - Error handling - Redirect logic **Dependencies:** react, react-router-dom, AuthContext **Exports:** Login component

Client/src/pages/Register/Register.jsx **Purpose:** User registration page **Key Functions:** - Registration form handling - Form validation - API integration - Success handling **Dependencies:** react, react-router-dom, AuthContext **Exports:** Register component

Client/src/pages/Search/Search.jsx **Purpose:** Property search and listing page **Key Functions:** - Service data fetching - Filter application - Results display - Modal integration **Dependencies:** react, AuthContext, FilterSidebar, ListingCard, ServiceModal **Exports:** Search component

Client/src/pages/Payment/Payment.jsx **Purpose:** Multi-step payment processing page **Key Functions:** - Payment form management - Step navigation - API integration - Booking creation **Dependencies:** react, react-router-dom, AuthContext, payment components **Exports:** Payment component

Client/src/pages/Dashboard/Dashboard.jsx **Purpose:** User dashboard with multiple views **Key Functions:** - View management - Data fetching - User profile display - Navigation handling **Dependencies:** react, AuthContext, dashboard view components **Exports:** Dashboard component

Client/src/pages/ProvideService/ProvideService.jsx **Purpose:** Property listing creation page **Key Functions:** - Multi-step form handling - Image upload - Form validation - Service creation API **Dependencies:** react, react-router-dom, AuthContext **Exports:** ProvideService component

Client/src/components/Header/Header.jsx **Purpose:** Main navigation header **Key Functions:** - Navigation menu - User authentication display - Mobile responsiveness **Dependencies:** react, react-router-dom, AuthContext **Exports:** Header component

Client/src/components/ListingCard/ListingCard.jsx **Purpose:** Property listing display card **Key Functions:** - Property information display - Action button handling - Wishlist integration - Navigation to payment **Dependencies:** react, react-router-dom **Exports:** ListingCard component

Client/src/components/ServiceModal/ServiceModal.jsx **Purpose:** Detailed property information modal **Key Functions:** - Detailed property display - Image gallery - Contact information - Modal controls **Dependencies:** react **Exports:** ServiceModal component

Client/src/components/FilterSidebar/FilterSidebar.jsx **Purpose:** Search filter interface **Key Functions:** - Filter option display - Filter state management - Real-time filtering **Dependencies:** react **Exports:** FilterSidebar component

=====

5.9 10. FRONTEND-BACKEND CONNECTION FLOW

5.9.1 10.1 API Communication Pattern

Standard Request Flow: 1. Frontend component initiates action (user interaction) 2. Component calls `fetch()` with appropriate endpoint 3. Request includes authentication cookies automatically 4. Backend middleware verifies authentication 5. Route handler processes request 6. Database operations performed if needed 7. Response sent back to frontend 8. Frontend updates UI based on response

5.9.2 10.2 Authentication Integration

Cookie-Based Authentication: - JWT tokens stored in HTTP-only cookies - Automatic inclusion in all requests - No manual token management in frontend - Secure against XSS attacks

AuthContext Integration: - Global authentication state - Automatic login status checking - Protected route handling - User data availability across components

5.9.3 10.3 Error Handling

Frontend Error Handling: - Try-catch blocks around API calls - User-friendly error messages - Loading state management - Fallback UI for errors

Backend Error Handling: - Consistent error response format - HTTP status codes - Detailed error messages - Logging for debugging

5.9.4 10.4 Data Transformation

Backend to Frontend: - MongoDB ObjectId to string conversion - Date formatting - Price formatting - Image URL construction

Frontend to Backend: - Form data serialization - Date object conversion - File upload handling - Validation before sending

=====

5.10 11. CORE FUNCTIONALITIES

5.10.1 11.1 User Management

Registration and Authentication: - Secure user registration with validation - JWT-based authentication system - Password hashing and security - Profile management capabilities

User Roles: - Regular users: Can search, book, and manage bookings - Property owners: Can list properties and manage listings - Admins: Can manage all aspects of the platform

5.10.2 11.2 Property Management

Property Listing: - Multi-step property creation form - Image upload and management - Detailed property information - Availability calendar

Property Search: - Advanced filtering options - Location-based search - Price range filtering - Amenity-based filtering

5.10.3 11.3 Booking System

Booking Process: - Date availability checking - Conflict prevention - Booking confirmation system - Status tracking

Booking Management: - User booking history - Booking status updates - Cancellation handling - Owner booking management

5.10.4 11.4 Payment Processing

Payment Methods: - Multiple payment options - Secure payment processing - Payment status tracking - Refund handling

Financial Management: - Payment history tracking - Transaction records - Revenue analytics - Fee calculation

5.10.5 11.5 Wishlist System

Wishlist Management: - Save favorite properties - Easy access to saved items - Wishlist sharing capabilities - Notification for saved items

=====

5.11 12. MONGODB COLLECTIONS AND SCHEMA

5.11.1 12.1 Collection Initialization

Location: Models are defined in server/models/ directory **Initialization:** Mongoose automatically creates collections when first document is inserted **Connection:** Established in server/server.js using mongoose.connect()

5.11.2 12.2 Collection Details

12.2.1 users Collection Schema File: server/models/User.js **Purpose:** Store user account information and profiles **Key Fields:** - `_id`: MongoDB ObjectId (auto-generated) - `name`: User's full name - `email`: Unique email address - `password`: Hashed password - `role`: User role (user/admin) - `profile`: Nested object with personal information - `createdAt`, `updatedAt`: Timestamps

Indexes: - `email`: Unique index for fast lookup - `createdAt`: For sorting and analytics

12.2.2 services Collection Schema File: server/models/Service.js **Purpose:** Store property listing information **Key Fields:** - `_id`: MongoDB ObjectId (auto-generated) - `title`: Property title - `propertyType`: Type of property - `location`: Nested object with address details - `price`: Monthly rent amount - `availability`: Date range and status - `propertyDetails`: Bedrooms, bathrooms, etc. - `amenities`: Available facilities - `owner`: Reference to users collection - `createdAt`, `updatedAt`: Timestamps

Indexes: - `location.district`: For location-based queries - `location.area`: For area-specific searches - `price`: For price range filtering - `owner`: For owner's listings

12.2.3 bookings Collection Schema File: server/models/Booking.js **Purpose:** Store booking requests and status **Key Fields:** - `_id`: MongoDB ObjectId (auto-generated) - `service`: Reference to services collection - `user`: Reference to users collection - `startDate`, `endDate`: Booking period - `totalAmount`: Total booking cost - `status`: Booking status - `paymentStatus`: Payment status - `confirmationCode`: Unique booking code - `createdAt`, `updatedAt`: Timestamps

Indexes: - `user`: For user's booking history - `service`: For property booking history - `startDate`, `endDate`: For availability checking

12.2.4 payments Collection Schema File: server/models/Payment.js **Purpose:** Store payment transaction records **Key Fields:** - `_id`: MongoDB ObjectId (auto-generated) - `booking`: Reference to bookings collection - `user`: Reference to users collection - `service`: Reference to services collection - `amount`: Payment amount - `paymentMethod`: Method used for payment - `status`: Payment status - `paymentDetails`: Method-specific details - `createdAt`, `updatedAt`: Timestamps

Indexes: - `user`: For user's payment history - `booking`: For booking-payment relationship - `status`: For status-based queries - `createdAt`: For chronological sorting

5.11.3 12.3 Data Relationships

Reference Relationships: - User → Services (One-to-Many) - User → Bookings (One-to-Many) - User → Payments (One-to-Many) - Service → Bookings (One-to-Many) - Booking → Payment (One-to-One)

Population Usage: - Services populated with owner information - Bookings populated with user and service details - Payments populated with booking and user information

5.11.4 12.4 Database Operations

Common Operations: - CRUD operations for all collections - Complex queries with filtering and sorting - Aggregation pipelines for analytics - Population for related data - Indexing for performance optimization

Performance Considerations: - Proper indexing on frequently queried fields - Pagination for large result sets - Efficient query patterns - Connection pooling - Query optimization

=====

5.12 CONCLUSION

This comprehensive analysis covers the complete architecture and implementation of the NestUp BD platform. The system demonstrates a well-structured full-stack application with proper separation of concerns, secure authentication, robust data modeling, and user-friendly interfaces. The modular design allows for easy maintenance and future enhancements while providing a solid foundation for a property rental platform.

The codebase follows modern web development best practices with React for the frontend, Node.js/Express for the backend, and MongoDB for data persistence. The authentication system ensures security, while the payment system provides multiple options for users. The comprehensive API design enables smooth communication between frontend and backend components.

This documentation serves as a complete reference for understanding, maintaining, and extending the NestUp BD platform.